IIIT DHARWAD · CLOUD COMPUTING · SEMESTER VII PROJECT REPORT

KubeAttackViz

From static coefficients to learned edge weights: machine-learning risk scoring for Kubernetes attack paths

Kubernetes v1.37 · three live kind clusters · 361,825 NVD vulnerabilities · CISA Known Exploited Vulnerabilities catalog 2026.11.09

SUBMITTED BY

Ganesh Daware	23BCS038
Gagan Rathod	23BCS106
Venkatesh Dhangar	23BCS041
Advait Pathak	23BCS094

SUBMITTED TO

Dr. Malay Kumar

Indian Institute of Information Technology (IIIT) Dharwad

November 2026

Abstract

KubeAttackViz models a Kubernetes cluster as a directed attack graph — each edge is a step an attacker could take — and uses four graph algorithms to answer a defender's questions: what a compromised component can reach (breadth-first search), the most likely route to each high-value target (Dijkstra), privilege loops (Johnson's algorithm) and the single fix that removes the most attack surface (graph surgery). The first version of the tool, presented earlier in the course, weighted every edge with **predefined static coefficients** chosen following the risk-assessment approach of *Stay at the Helm* [1]. Following the evaluator's feedback, we replaced those coefficients with **probabilities learned from data**.

Edges whose feasibility is fully decided by the cluster's configuration now receive probability ≈ 1 from an exact feasibility engine; edges that require an exploitable vulnerability use a gradient-boosted model trained on **361,825 NVD vulnerability records** labelled with the CISA Known Exploited Vulnerabilities catalog. On vulnerabilities published in 2024 or later, the learned model raises ROC-AUC from **0.770** (CVSS score, as used by the static coefficients) to **0.887**, average precision from **0.012** to **0.108** (×8.8), and precision among the 100 highest-ranked vulnerabilities from **5%** to **54%**. The feasibility engine agrees with a real Kubernetes API server on all 21,019 RBAC checks of 26 randomized lab scenarios, and path rankings are stable under the one parameter that cannot be learned.

Contents

Abstract 2

Contents 3

1 Introduction and background 4

1.1 The problem . . . 4

1.2 First presentation and evaluator feedback . . . 4

1.3 What this report covers . . . 4

2 System architecture 4

3 Attack graph and the four core algorithms 5

3.1 Building the graph . . . 5

3.2 Blast radius — breadth-first search . . . 6

3.3 Most likely attack path — Dijkstra . . . 6

3.4 Privilege loops — Johnson's algorithm . . . 6

3.5 Chokepoints — graph surgery . . . 6

4 Before: predefined static coefficients 7

4.1 Limitations of the static scheme . . . 8

5 After: learned edge weights 8

5.1 From effort to probability . . . 8

5.2 Dataset and labels . . . 9

5.3 Features and leakage guards . . . 9

5.4 Algorithm . . . 9

6 Before vs after: comparison 9

6.1 Same CVSS score, very different reality . . . 9

6.2 Predictive performance on future vulnerabilities . . . 10

6.3 Effect on the attack graphs . . . 11

7 Sensitivity analysis and validation 13

7.1 Sensitivity to the one assumed parameter . . . 13

7.2 Validation against a real Kubernetes cluster . . . 14

8 Live system, explanations and alerts 14

8.1 Three live clusters . . . 14

8.2 Natural-language explanations . . . 15

8.3 Temporal analysis and alerts . . . 15

8.4 Other scores . . . 15

9 Limitations and future work 15

10 Conclusion 16

References 16

Reproducibility . . . 16

1 Introduction and background

1.1 The problem

Kubernetes clusters are rarely breached by a single dramatic flaw. Breaches are chains: an internet-facing pod running an outdated image, a ServiceAccount token mounted by default, a RoleBinding that grants more than intended, a secret that unlocks a database. Each link looks acceptable in isolation; the risk lives in the path. A useful tool must therefore reason about paths, rank them by how likely and how damaging they are, and tell the defender which single change helps most.

1.2 First presentation and evaluator feedback

The first version of KubeAttackViz was presented with a working command-line tool and a browser visualiser: it ingested a cluster (or a JSON description of one), built the attack graph, ran the four algorithms and produced a kill-chain report with remediation. The work was well received. The evaluator's main question was about the **edge weights**: they were predefined constants, so the ranking of attack paths depended on numbers chosen by hand. The suggestion was to **learn the weights with a machine-learning model** instead.

1.3 What this report covers

- the system and its four core algorithms (Sections 2–3);
- the original weighting with static coefficients — the **before** state (Section 4);
- the upgrade: an exact feasibility engine plus an ML model for exploitation probability — the **after** state (Section 5);
- a quantitative before/after comparison with graphs (Section 6), sensitivity analysis and validation against a real cluster (Section 7);
- the live multi-cluster demonstration, explanations and alerts (Section 8), limitations and conclusions (Sections 9–10).

2 System architecture

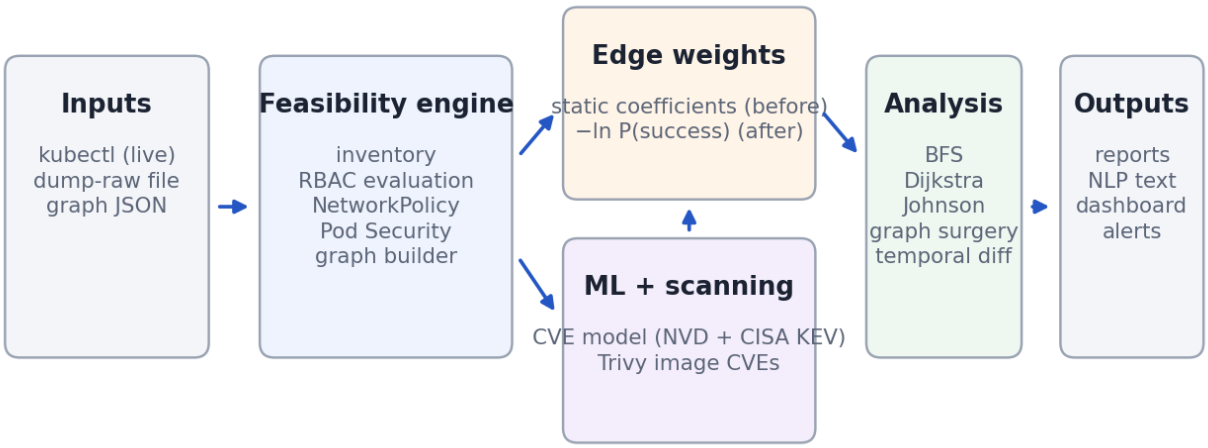


Figure 1. Data flow. All cluster access is read-only (get/list); Secret values are never fetched, only names and types.

Inputs are a live cluster (through `kubectl`), a raw dump of one, or a hand-written graph. The **feasibility engine** evaluates Kubernetes RBAC, NetworkPolicy and Pod Security semantics to decide which attack steps exist. The **weighting layer** turns each step into a cost — the static coefficients (before) or `−ln P(success)` (after) — using the ML model and container-image scanning with Trivy. The **analysis layer**

runs the algorithms, a temporal diff between snapshots, and a natural-language explainer, and serves results to reports, the terminal and a live dashboard.

Component	Responsibility	Main files
Ingestion	kubectl, raw dumps and graph JSON → typed resources	ingestion.py, k8s/inventory.py
Feasibility engine	RBAC, NetworkPolicy, Pod Security, graph edges with evidence	k8s/rbac.py, netpol.py, podsec.py, graph.py
Weights	static coefficients or −ln P from the ML model	graph_builder.py, ml/weights.py
ML	dataset download, features, training, inference	ml/datasets.py, cve_features.py, cve_model.py
Algorithms	BFS, Dijkstra, Johnson cycles, graph surgery	algorithms/*.py
Temporal / live	snapshots, diffs, alerts; multi-cluster monitor	temporal.py, live.py
Outputs	reports, NLP explanations, dashboard	report_generator.py, nlp_explainer.py, visualizer/

3 Attack graph and the four core algorithms

3.1 Building the graph

An edge is created only when the cluster's configuration allows the step. Edges carry the **evidence** that makes them possible and a **feasibility** label: *exact* when configuration fully decides the step, *conditional* when the attempt is permitted but success depends on something configuration cannot decide — typically an exploitable vulnerability in the target workload.

Edge	Created when	Feasibility
internet → pod	a LoadBalancer/NodePort Service or an Ingress routes to the pod	conditional
pod → ServiceAccount	the token is auto-mounted (pod setting overrides SA setting)	exact
identity → binding → role	subject match, incl. system:serviceaccounts[:ns] groups	exact
binding → secret / pod / SA / node / admin	RBAC grants read-secrets, exec, create pods, create tokens, impersonate, bind/escalate or nodes/proxy, with namespace scope, wildcards and resourceNames	exact
pod → secret	volume or environment reference	exact
pod → pod	NetworkPolicy allows ingress at the destination and egress at the source	conditional
pod → node	the pod violates a Pod Security Standards baseline host-isolation control	exact if privileged

Entry points are the internet, every human User named in an RBAC binding (a compromised credential) and pods labelled as entry points (e.g. a CI runner executing third-party code). Targets are Secrets, Nodes and cluster-admin-equivalent roles. The graph is scoped to application namespaces and pruned to nodes that can lie on an attack path — the demo clusters shrink from about 185 to 15–20 nodes without losing any path.

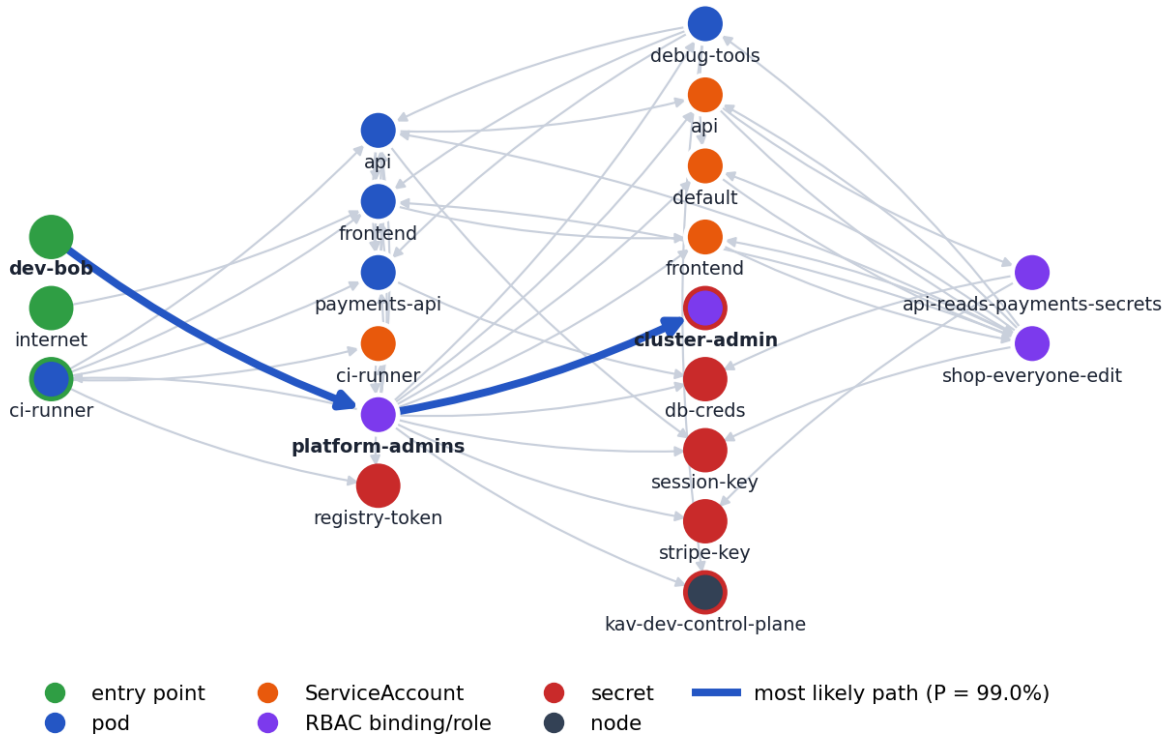


Figure 2. The real attack graph of the kav-dev cluster (20 nodes), laid out by hop distance from the entry points. Highlighted: the most likely attack path, dev-bob → platform-admins → cluster-admin.

3.2 Blast radius — breadth-first search

From an origin s, BFS visits nodes in order of hop distance and records each at its first (shortest) distance, so every node appears in exactly one layer. It is computed for every entry point and every workload and ranked by the number of targets reached. Complexity O(|V|+|E|) per origin. On kav-dev the developer credential dev-bob reaches 6 targets across 18 nodes within three hops; the internet reaches 2.

3.3 Most likely attack path — Dijkstra

For each pair of entry point s and target t the minimum-weight path is

$$\pi^*(s,t) = \arg\min_{\pi:\, s \rightsquigarrow t} \sum_{e \in \pi} w(e)$$

computed in O((|V|+|E|) log |V|) per source. With the learned weights of Section 5 this is exactly the most probable path.

3.4 Privilege loops — Johnson's algorithm

Elementary cycles are enumerated with Johnson's algorithm and deduplicated by rotating each cycle to start at its smallest node. Real RBAC graphs can contain an astronomical number of cycles (cluster-admin can create a pod as any ServiceAccount, which loops back), so enumeration is bounded to cycles of at most 8 hops and at most 200 cycles; the report states when it stopped early. This bound was a necessary fix found on the first live cluster.

3.5 Chokepoints — graph surgery

For every intermediate node v, the node is removed logically and the loss of attack surface is measured:

$$\Delta(v) = N(G) - N(G \setminus v)$$

N counts simple source-to-target paths of at most 15 edges when this fits a budget of 20,000 search expansions (exact), and reachable (source, target) pairs otherwise (polynomial). The node with the largest Δ is the best single fix.

Rank	Chokepoint on kav-dev	Paths removed	Share
1	platform-admins	552 / 655	84%
2	api	523 / 655	80%
3	shop-everyone-edit	507 / 655	77%

4 Before: predefined static coefficients

The first version assigned every relationship a fixed effort coefficient and every resource type a fixed impact and likelihood, following the risk-assessment approach of *Stay at the Helm* [1], which characterises the nodes and edges of a deployment's topology graph with security features and extracts the riskiest attack paths from the resulting scores. Lower edge weight means an easier step.

Relationship	Edge weight w(e)	Resource	Impact I
runs_as	2.0	database	10.0
connects_to	2.0	secret	9.0
exposes	2.5	node	7.0
binds_to	3.0	clusterrole	6.5
uses	3.0	clusterrolebinding	6.0
reads	3.0	serviceaccount	5.0
mounts	3.5	role	4.5
grants	4.0	pod	4.0
writes	4.0	ingress	4.0
has_secret	4.5	rolebinding	3.5
accesses	5.0	service	3.0
escalates_to	6.0	configmap	3.0
		namespace	1.0

Likelihood coefficients (pod 2.0, service 3.0, ServiceAccount 2.0, role 1.0, secret 1.5, database 1.0) were raised by fixed increments for observed risk factors: +3 for a privileged container or host networking, +2 for running as root, +3 for a LoadBalancer/NodePort service. Node risk and the CVE adjustment of an edge's weight were:

$$r_v = \frac{L_v}{10} \cdot I_v$$

$$w'(e) = \max\left(0.1,\ w(e)\left(1 - \frac{\text{CVSS}}{10}\right)\right)$$

and a path's risk combined its total cost C with the target's impact:

$$\text{risk}(\pi) = \frac{I_t}{1 + C(\pi)/10}, \qquad C(\pi) = \sum_{e \in \pi} w'(e)$$

4.1 Limitations of the static scheme

- **Hand-chosen numbers.** Changing runs_as from 2.0 to 3.0 can reorder every path; nothing in the data justifies either value.
- **CVSS is not exploitation.** Two vulnerabilities with CVSS 9.8 receive the same reduction even when one is used by attackers daily and the other never; Figure 3 shows how large this difference is.
- **No distinction between certain and uncertain steps.** Using a permission that RBAC grants is certain once a credential is held; entering a workload needs an exploit. Both received comparable effort weights.
- **No probabilistic meaning.** Path cost was an abstract sum, so 'how likely is this attack?' could not be answered.

Note on severity: the first version also labelled a path more severe when its summed weight was higher, i.e. when it was *harder* for the attacker. This inversion was corrected as part of the upgrade; the 'before' results in this report use the corrected static formula above, so the comparison isolates the effect of the weights.

5 After: learned edge weights

5.1 From effort to probability

Each edge receives a probability P(e) that the attacker succeeds at that step, and the weight w(e) = −ln P(e). Because logarithms turn products into sums, minimising total weight maximises the probability that every step succeeds:

$$w(e) = -\ln P(e) \;\Rightarrow\; \sum_{e \in \pi} w(e) = -\ln \prod_{e \in \pi} P(e)$$

$$P(\pi) = e^{-C(\pi)}, \qquad \text{risk}(\pi) = I_t \cdot P(\pi)$$

Risk becomes an expected impact on a 0–10 scale (CRITICAL ≥ 4, HIGH ≥ 1.5, MEDIUM ≥ 0.5). Probabilities are clipped to [0.01, 0.995]. Each edge reports the **basis** of its probability:

Basis	When	P(e)
exact-config	configuration decides the step (RBAC grant, mounted token, routing, privileged pod)	0.995
cve-model	conditional step and a CVE is known on the target workload	1 − (1 − p_u)(1 − p_x)
scenario-param	conditional step, no known CVE	p_u = 0.05 (assumption)

$$P(e) = 1 - (1 - p_u)\,(1 - p_x)$$

p_x is the **learned** probability that the CVE is exploited in the wild. p_u is the chance of crossing with no known vulnerability; no public dataset measures it, so it is an explicit assumption whose influence is measured in Section 7.1. A known CVE is treated as an independent second way across, so it can only raise P(e).

5.2 Dataset and labels

Split	CVEs	Exploited (KEV)	Base rate
Train — published up to 2023	179,970	1,174	0.65%
Test — published 2024 or later	181,855	569	0.31%
All years (final model)	361,825	1,743	0.48%

Features come from NVD records [2] (yearly feeds via the FKIE mirror [3]); the label is membership of the CISA Known Exploited Vulnerabilities catalog [4]. The split is temporal — training on the past, testing on the future — which is what a deployed model faces. Recent CVEs have had less time to be added to the catalog, so the test base rate is lower and the evaluation conservative.

5.3 Features and leakage guards

- CVSS base metrics: attack vector, attack complexity, privileges required, user interaction, scope, confidentiality/integrity/availability impact; base, exploitability and impact scores (CVSS v2 as fallback).
- Weakness type (CWE); reference tags (public exploit, patch, vendor advisory) and their count; description keywords (remote code execution, authentication bypass, deserialization, path traversal, …); vendor popularity; publication year.

Leakage guards. Fields that are consequences of a CVE being added to the catalog are removed: the CISA fields in NVD records, references to cisa.gov, vulnStatus and lastModified. One remaining feature needs care: the description phrase 'exploited in the wild', which vendors sometimes add after exploitation is seen. Section 6.2 reports the model with and without it.

5.4 Algorithm

Gradient-boosted decision trees (scikit-learn HistGradientBoostingClassifier: log-loss, 300 iterations, 15 leaves, at least 100 samples per leaf, L2 = 3, early stopping). Each tree h_m corrects the log-odds residuals of the ensemble so far, minimising the log-loss:

$$F_m(x) = F_{m-1}(x) + \eta\, h_m(x), \qquad p_x = \sigma(F_M(x)) = \frac{1}{1+e^{-F_M(x)}}$$

$$\mathcal{L} = -\frac{1}{N}\sum_{i=1}^{N}\left[y_i \ln p_i + (1-y_i)\ln(1-p_i)\right]$$

Two baselines use the same split: logistic regression on the CVSS base score alone — the information the static coefficients used — and logistic regression on all features. Isotonic and sigmoid post-calibration were tested and rejected: they inflated predictions about five-fold and scored a worse Brier score than predicting the base rate. At analysis time, CVEs in the catalog get $p_x = 0.99$; known CVEs use their out-of-fold score (never an in-sample prediction); container images are scanned with Trivy [5] and each pod keeps its ten highest-ranked CVEs.

6 Before vs after: comparison

6.1 Same CVSS score, very different reality

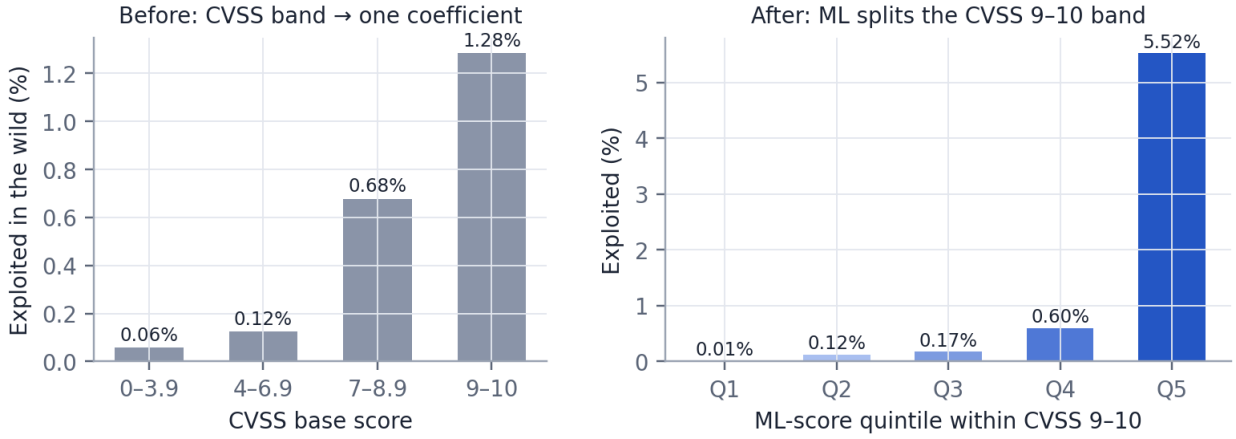


Figure 3. Left: observed exploitation rate per CVSS band — the static scheme gives all CVEs in a band the same treatment. Right: inside the CVSS 9–10 band (49,425 CVEs, 1.3% exploited), the ML score separates 0.01% (lowest fifth) from 5.5% (highest fifth).

CVSS measures how severe a flaw *would be*, not whether attackers use it. Among critical-rated CVEs only 1.3% appear in the catalog. The learned model ranks them: its top fifth is exploited 546× more often than its bottom fifth, and its top 1 % of critical CVEs has an exploitation rate of 41% — 32× the band average. A static coefficient cannot express this.

6.2 Predictive performance on future vulnerabilities

Model (test: CVEs from 2024 on)	ROC-AUC	PR-AUC	Brier	Prec@10	Prec@100	Recall@1000
CVSS score only (before)	0.770	0.012	0.00312	0%	5%	7%
Logistic regression	0.877	0.112	0.00297	80%	48%	24%
Gradient boosting (after, deployed)	0.887	0.108	0.00296	80%	54%	19%
GB without 'in the wild' text	0.876	0.097	0.00298	90%	42%	16%
FIRST EPSS (reference)	0.975	0.428	0.00303	100%	89%	51%

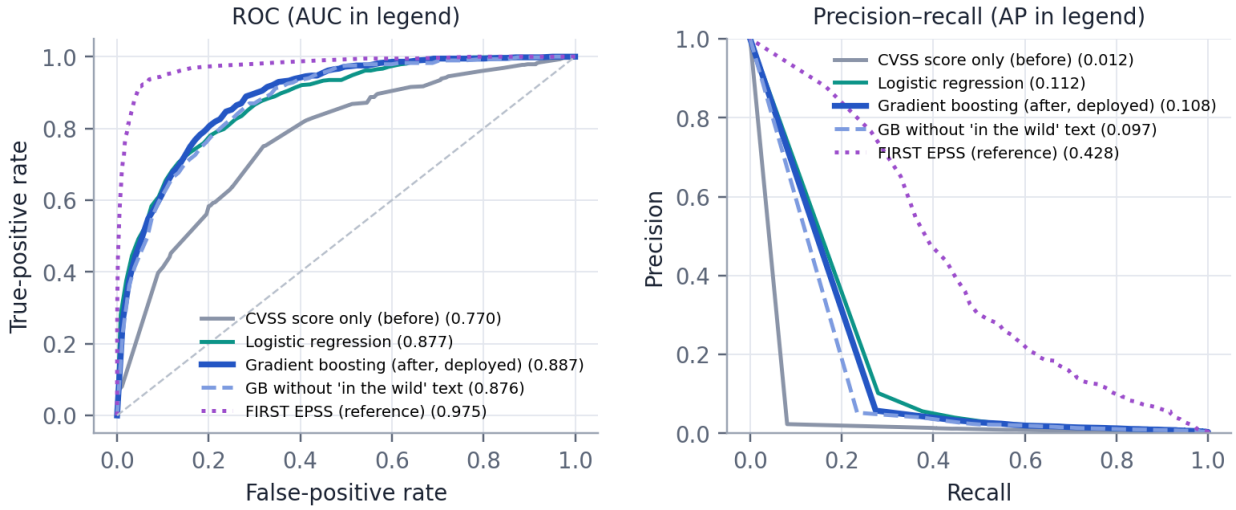


Figure 4. ROC and precision–recall curves on the 2024+ test set. With a 0.31% base rate, precision–recall is the more informative view: the CVSS-only curve stays near the floor.

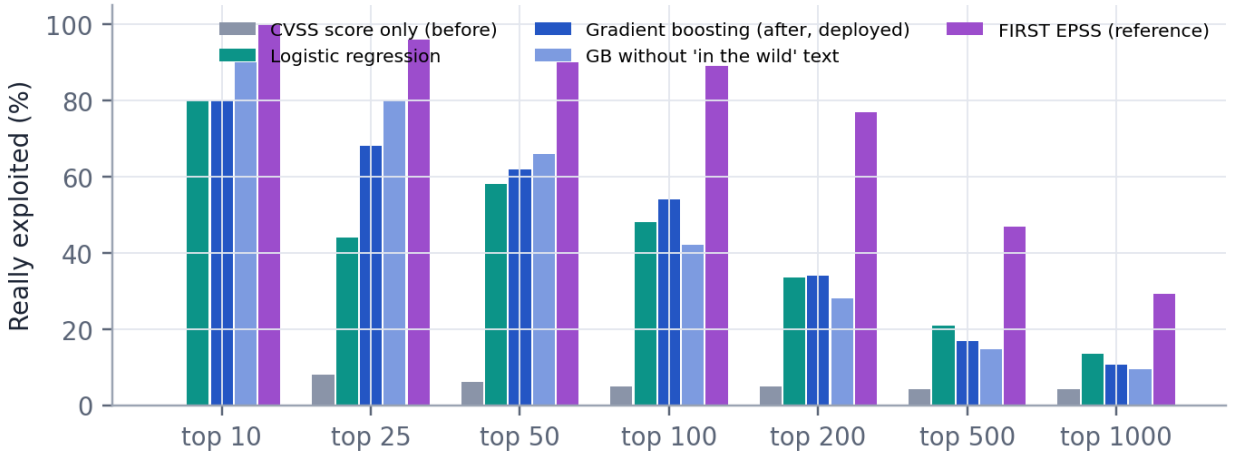


Figure 5. Precision at k: of the k CVEs each method ranks highest, the share that were really exploited — what an analyst triaging alerts experiences.

Without the 'in the wild' description phrase the model still reaches ROC-AUC 0.876 and precision@100 of 42%, so the improvement does not depend on that feature. FIRST's EPSS [6] (ROC-AUC 0.975) is shown only as a reference: it is a production system with additional threat-intelligence inputs, scored with today's data, so it has hindsight on this test set.

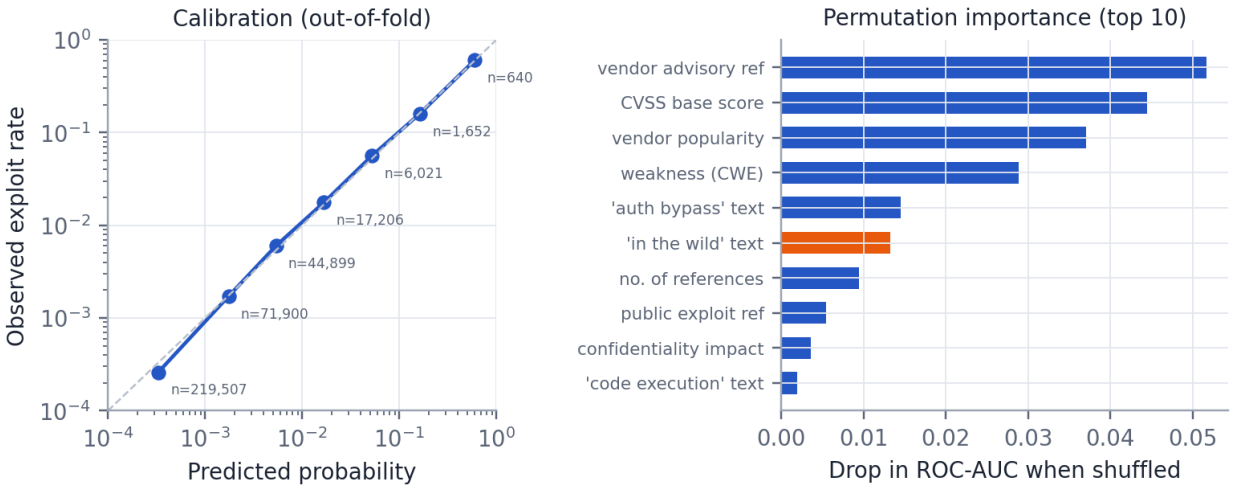


Figure 6. Left: calibration of the deployed model (out-of-fold, all years, log scales): predicted and observed rates agree in every band (out-of-fold ROC-AUC 0.936). Right: permutation importance on the test set.

6.3 Effect on the attack graphs

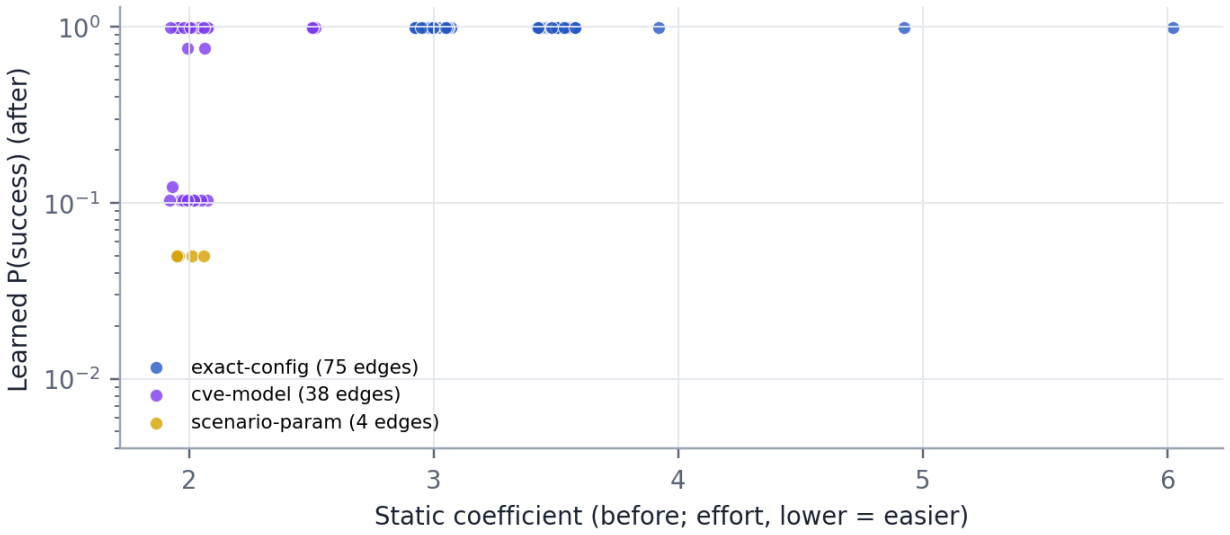


Figure 7. Every edge of the three live clusters: static coefficient (x) against learned probability (y). Static coefficients do not separate certain steps (blue, P = 0.995) from exploit-dependent ones (purple, P set by the CVE model) — the same coefficient maps to probabilities from below 0.01 to 0.995.

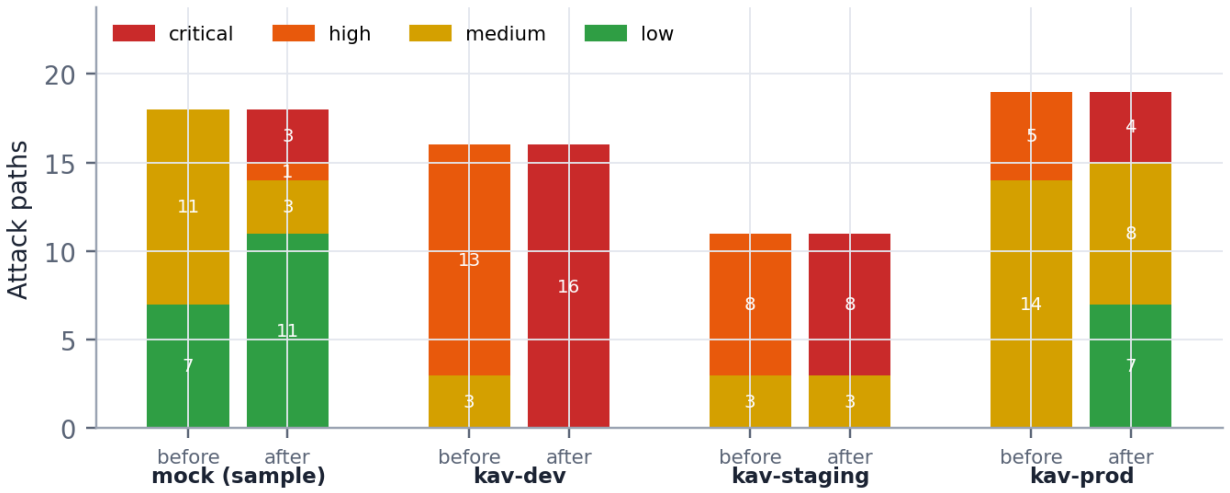


Figure 8. Severity of attack paths per cluster with static coefficients (before) and learned weights (after). Image CVEs from Trivy scans of the running pods.

Cluster	Nodes	Paths	Edge bases (after)	Spearman ρ before vs after
mock (sample JSON)	41	18	legacy-prior 48	0.05
kav-dev (live)	20	16	cve-model 9, exact-config 38, scenario-param 4	0.71
kav-staging (live)	19	11	cve-model 13, exact-config 25	0.84
kav-prod (live)	19	19	cve-model 16, exact-config 12	0.48

Rank correlation between the two orderings is moderate on the live clusters (ρ = 0.48–0.85) and close to zero on the hand-written sample graph: the learned weights substantially re-order the paths. Table 1 shows why on two clusters.

#	kav-dev — static coefficients (before)		learned weights (after)	
1	ci-runner → registry-token	HIGH risk 6.67	dev-bob → platform-admins → cluster-admin	CRITICAL P 99.0%
2	dev-bob → platform-admins → cluster-admin	HIGH risk 5.88	ci-runner → ci-runner → platform-admins → cluster-admin	CRITICAL P 98.5%
3	ci-runner → payments-api → db-creds	HIGH risk 5.81	ci-runner → registry-token	CRITICAL P 99.5%

#	kav-prod — static coefficients (before)		learned weights (after)	
1	ci-runner → registry-token	HIGH risk 6.67	ci-runner → registry-token	CRITICAL P 99.5%
2	ci-runner → frontend → api → session-key	HIGH risk 5.14	sre-carol → sre-carol-breakglass → ledger → ledger-creds	CRITICAL P 98.5%
3	ci-runner → frontend → checkout → checkout-signing-key	HIGH risk 5.14	sre-carol → sre-carol-breakglass → payments-api → db-creds	CRITICAL P 98.5%

Table 1. Top three attack paths before and after on kav-dev and kav-prod.

With static coefficients a path that crosses several permission hops looks expensive. With learned weights those hops are certain once the credential is held, so compromised-credential paths rise to the top — consistent with how real cluster breaches unfold — and paths from the internet are bounded by the exploitation probability of the first workload's CVE. On kav-dev the frontend image carries catalogued vulnerabilities, so the internet path is near-certain; on kav-prod the internal API's best CVE has $p_x = 0.057$, which caps the internet path at about 10%.

7 Sensitivity analysis and validation

7.1 Sensitivity to the one assumed parameter

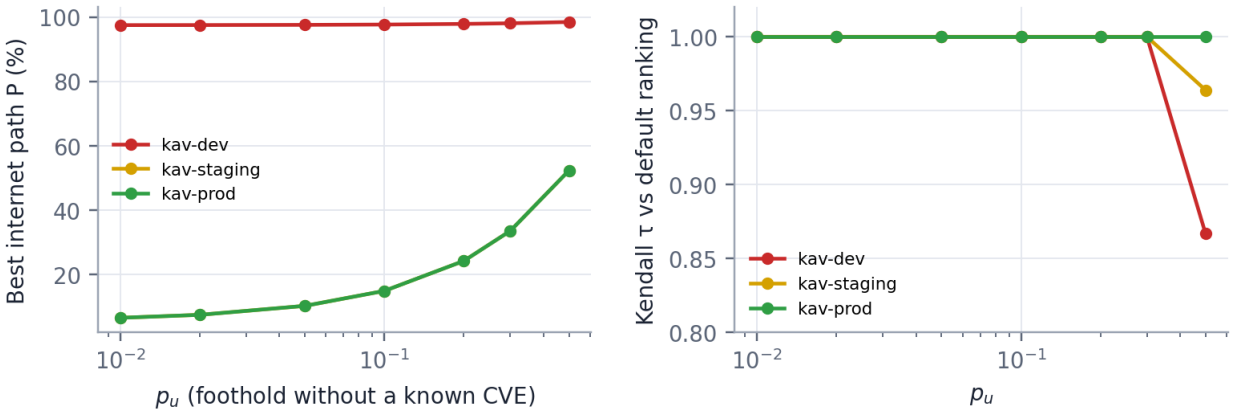


Figure 9. Left: probability of the best internet path as p_u varies from 0.01 to 0.5. Right: Kendall's τ between each ranking and the default (p_u = 0.05).

Path rankings are identical ($\tau = 1$) on all three clusters for every value up to 0.3, and the top-ranked path never changes; only absolute probabilities move (on kav-prod the best internet path goes from 6.5% to 52% across the range). The decisions the tool supports — which path to fix first, which chokepoint to remove — are therefore robust to the only quantity the model cannot learn.

7.2 Validation against a real Kubernetes cluster

A generator produced 26 random but valid configurations (RBAC rules with wildcards, subresources and resourceNames; group and cross-namespace subjects; default-deny, AND/OR and port-specific NetworkPolicies; token automount at pod and ServiceAccount level) on a local Kubernetes v1.37 cluster. Every label was **measured**: RBAC with a SubjectAccessReview — the API server's own authorizer — network reachability with an HTTP request executed inside the source pod, and token mounting from the volume injected by admission.

Check	N	Precision	Recall	F1
RBAC (SubjectAccessReview)	21,019	1.000	1.000	1.000
Token mounting	190	1.000	1.000	1.000
Network, no host-network pod	1,050	1.000	1.000	1.000
Network, host-network pod involved	166	0.873	0.605	0.715
All checks	22,425	0.996	0.982	0.989

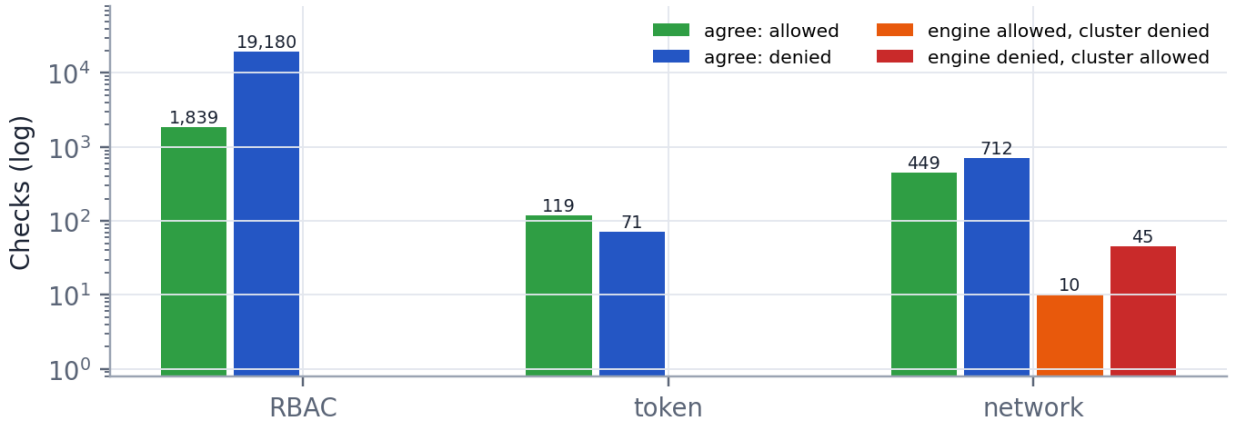


Figure 10. Confusion counts per check type (log scale).

The engine matched the API server on every RBAC and token check and on every network check that did not involve a host-network pod. All 55 disagreements involve pods with hostNetwork: true — NetworkPolicy does not select such pods and their traffic leaves from the node's IP — a documented limitation with a clear fix (Section 9).

8 Live system, explanations and alerts

8.1 Three live clusters

Cluster	Posture	Entry points	Nodes	Paths
kav-dev	loose: cluster-admin for CI and a developer, privileged debug pod, no NetworkPolicies	internet, ci-runner, dev-bob	20	16
kav-staging	mixed: payments NetworkPolicy, CI reads every secret, on-call edit	internet, ci-runner, qa-alice, sre-carol	19	11
kav-prod	hardened: Pod Security restricted, default-deny, no token mounts, break-glass exec	internet, ci-runner, sre-carol	19	19

The clusters are real Kubernetes (kind [7]): API server, scheduler, kubelet, containerd and network plugin, running real container images. A monitor thread per cluster scans every 15 seconds — kubectl, Trivy (cached), learned weights and all analyses take about 3–5 seconds — and publishes to a dashboard that refreshes when a cluster changes. An interactive terminal menu lists the clusters, shows pods and RBAC,

runs the step-by-step ML analysis and starts the live monitor. A typical explanation of one path, as printed by the monitor:

```
> #2 CRITICAL risk 8.87/10 P(path)=98.5% sre-carol → sre-carol-breakglass → ledger →
ledger-creds
  1. sre-carol -[bound_to]-> sre-carol-breakglass  P=0.995  config fact
  2. sre-carol-breakglass -[can_exec]-> ledger  P=0.995  config fact
  3. ledger -[mounts]-> ledger-creds  P=0.995  config fact
```

8.2 Natural-language explanations

Explanations are generated locally without an external language model. The command-line explainer maps each relationship to a verb phrase and each resource type to a noun phrase and produces an opening sentence, attack categories, a step-by-step chain with CVE context and remediation. The dashboard's query bar ('How can internet reach production-db?') detects one of five intents by comparing words with keyword lists using a Levenshtein-based similarity, and resolves the resource named in the question:

$$\mathrm{sim}(a, b) = 1 - \frac{\mathrm{lev}(a, b)}{\max(|a|, |b|)}$$

8.3 Temporal analysis and alerts

Snapshots are compared per (source, target) reachability rather than per exact route, so a re-route is not reported as a new plus an eliminated path. Alerts cover new attack paths, existing targets that became easier to reach, new privilege edges, new high-value resources, removed controls and attack-surface growth, each with a causal attribution naming the added resources and links. In the live demonstration, binding a production ServiceAccount to cluster-admin raised a HIGH alert within one scan; because that pod does not mount its token, no new attack path was reported — the dangerous grant is flagged without inventing a path that does not exist.

8.4 Other scores

$$r'_v = r_v\,(1 + \log_2(1 + k_v))$$

Node risk amplification raises a node's base risk r_v by the number k_v of attack paths through it; an advanced path score combines risk and the highest CVSS on the path (0.7 · risk + 0.3 · max CVSS).

9 Limitations and future work

- **Host-network pods.** NetworkPolicy semantics for hostNetwork pods are not modelled (all 55 lab disagreements). Fix: treat them as unselectable by policies and match their traffic only by node addresses.
- **The no-CVE foothold probability p_u** is assumed. Rankings are insensitive to it, but absolute probabilities scale with it; outcomes of real penetration tests could replace it.
- **Label definition.** The CISA catalog is a lower bound on exploitation, so learned probabilities are conservative; recent CVEs suffer label lag.
- **Image scanning** matches OS and language packages; software compiled from source inside an image is not identified.
- **Not modelled:** admission webhooks, ipBlock peers, non-resource URLs; aggregated ClusterRoles are read as materialised by the API server.
- **Future evaluation:** a direct comparison with the scoring of [1] on the same charts, and path-ranking metrics (MRR, NDCG) on vulnerable-by-design clusters with documented attack paths.

10 Conclusion

Answering the evaluator's question, KubeAttackViz no longer depends on hand-chosen edge weights. Steps that configuration decides are now computed exactly — and checked against a real API server on 21,019 RBAC decisions — while steps that need an exploit use a probability learned from 361,825 real vulnerabilities and their observed exploitation. Against the information the static coefficients used, the learned model ranks future vulnerabilities far better (ROC-AUC 0.770 → 0.887; precision of the top 100 5% → 54%), its probabilities are calibrated, and the resulting path rankings are stable under the one assumption that remains. The weights now have a meaning — the probability that an attacker succeeds — which lets the tool say not only which path is worst, but how likely it is and which single step carries the uncertainty.

References

[1] A. Blaise and F. Rebecchi, "Stay at the Helm: secure Kubernetes deployments via graph generation and attack reconstruction," in *Proc. IEEE International Conference on Cloud Computing (CLOUD)*, 2022. doi:10.1109/CLOUD55607.2022.00022. https://hal.science/hal-03705672

[2] NIST, National Vulnerability Database. https://nvd.nist.gov/

[3] Fraunhofer FKIE, NVD JSON data feeds (mirror). https://github.com/fkie-cad/nvd-json-data-feeds

[4] CISA, Known Exploited Vulnerabilities catalog, version 2026.11.09. https://www.cisa.gov/known-exploited-vulnerabilities-catalog

[5] Aqua Security, Trivy vulnerability scanner. https://github.com/aquasecurity/trivy

[6] FIRST, Exploit Prediction Scoring System (EPSS). https://www.first.org/epss/

[7] Kubernetes SIGs, kind — Kubernetes in Docker. https://kind.sigs.k8s.io/

[8] Kubernetes documentation: RBAC, Network Policies, Pod Security Standards, Authorization (SubjectAccessReview). https://kubernetes.io/docs/

[9] D. B. Johnson, "Finding all the elementary circuits of a directed graph," *SIAM Journal on Computing*, 4(1), 1975.

[10] E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numerische Mathematik*, 1, 1959.

[11] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *JMLR*, 12, 2011.

Reproducibility

```
pip install -r requirements.txt
python main.py run-tests    # 31 built-in tests
python main.py ml-fetch-data && python main.py ml-train
python main.py lab-run --scenarios 25 --seed 7 && python main.py lab-report
./demo/setup.sh && ./demo/present.sh
```

Every dataset download is logged with its URL, size and SHA-256. The trained model, the catalog snapshot and the out-of-fold scores ship with the source code, so every number and figure in this report can be regenerated.